DeployGuard Native Deployment Architecture

Why we could not implement our module reliably, even when many individual parts were good

This report documents our own DeployGuard Native Deployment module. We could not implement it to the reliability level required for a production deployment platform because its output depended on a long chain of inferred decisions. One wrong call at an early level could become authoritative and disrupt every correct stage after it.

Executive summary

We could not complete our native deployment module as a consistently reliable end-to-end system, not because every individual component was poor, but because too many components depended on inferred facts from earlier stages. The module tried to identify repository structure, stack, framework, application units, topology, package strategy, commands, ports, environment ownership, database relationships, Docker strategy, and finally a canonical BuildPlan before AWS execution. Many of those decisions could be correct in isolation. The failure mode was propagation: when even one identification was wrong or a contract drifted between stages, downstream modules could faithfully execute the wrong assumption. That made the system fragile across the huge variety of real repositories and meant one incorrect call could outweigh many otherwise-correct decisions.

Core failure pattern

Repository fact -> detector decision -> normalized plan -> generated artifact -> AWS runtime.

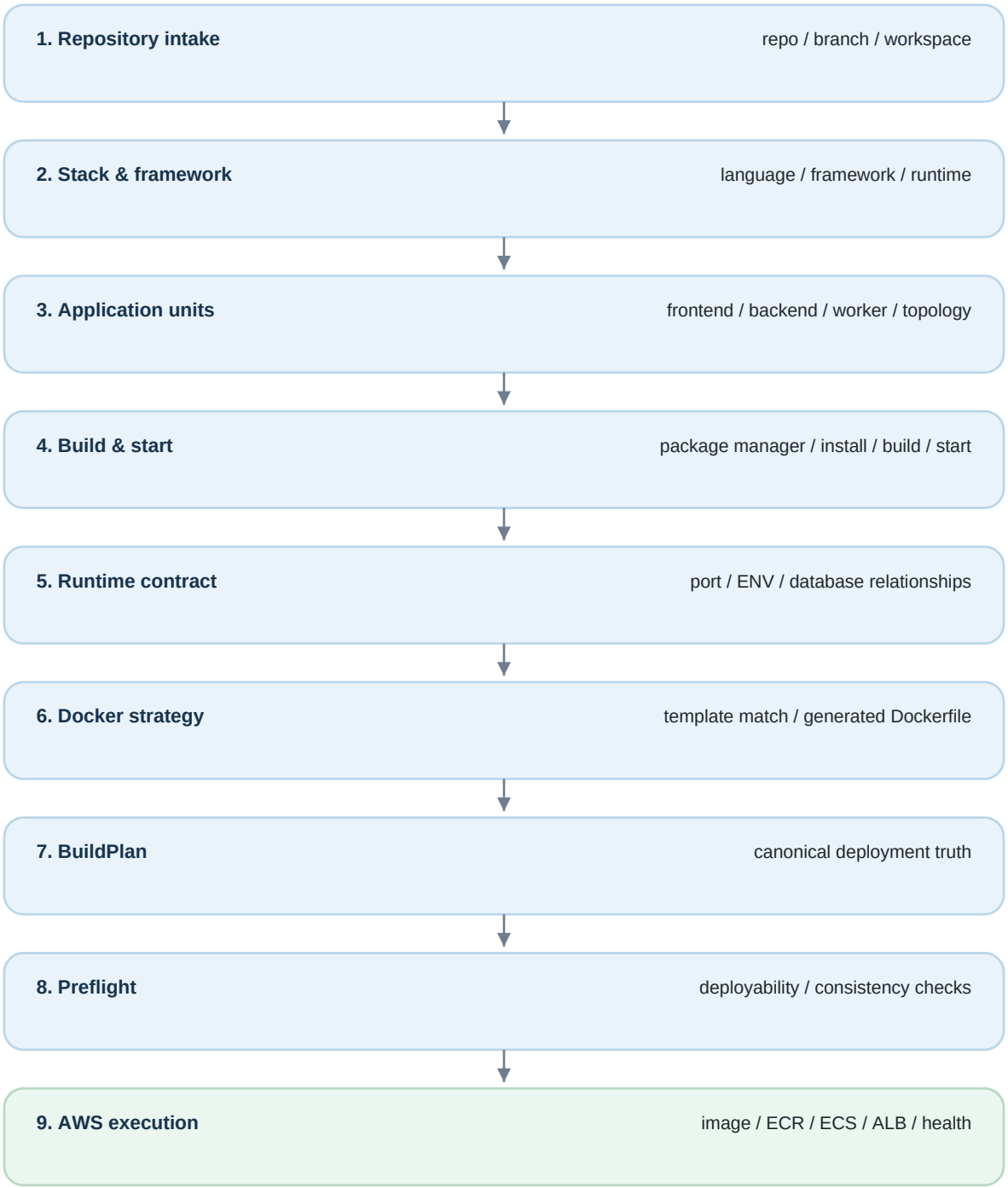
If one detector decision was wrong, the mistake was not isolated. It was copied into the next stage and could become authoritative deployment state.

What this report covers

- The full native architecture as a chain of identification levels.
- What each level tried to infer or normalize.
- Why good local decisions could still produce a bad end-to-end deployment.
- How a single wrong call could cascade across the pipeline.
- Why reducing interpretation errors was difficult even when individual stages were well engineered.

Architecture overview

The native system was a source-understanding pipeline. Every stage produced facts that the next stage consumed.

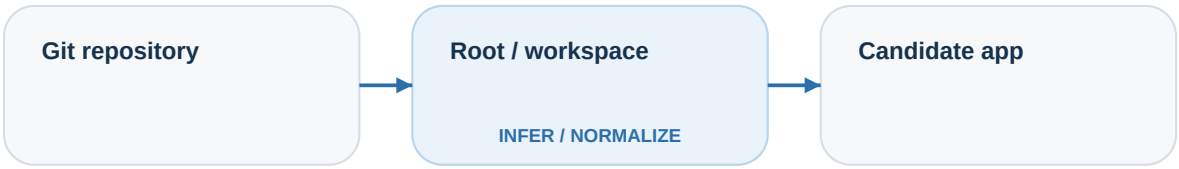


Important architectural property

This was not a set of independent checks. It was a dependency chain. The more fields the system inferred, the larger the blast radius of an incorrect early decision.

Levels 1-2: source and stack understanding

Level 1 - Repository and application-root resolution



What this level tried to determine

Determine the repository root, workspace root, application root, install root, and build context. This was the first attempt to decide which part of the repository actually represented the deployable application.

Risk at this level

If the wrong directory became authoritative, every later detector could inspect the wrong package files, commands, static assets, Docker context, or environment files.

Level 2 - Stack, language, framework, and runtime detection

What this level tried to determine

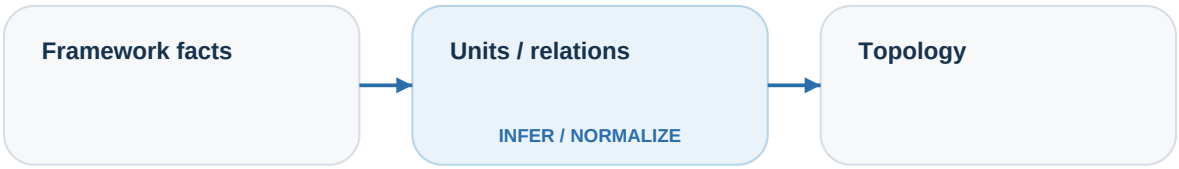
Identify language and framework signals such as Node, Python, Flask, Django, React, NestJS, package metadata, runtime versions, and the likely execution model. Representative services included framework and mainstream detector resolution.

Risk at this level

A wrong framework call could influence package-manager selection, build command, start command, expected files, generated Dockerfile template, default port assumptions, health behavior, and topology rules.

Levels 3-4: topology and execution planning

Level 3 - Application-unit and topology discovery



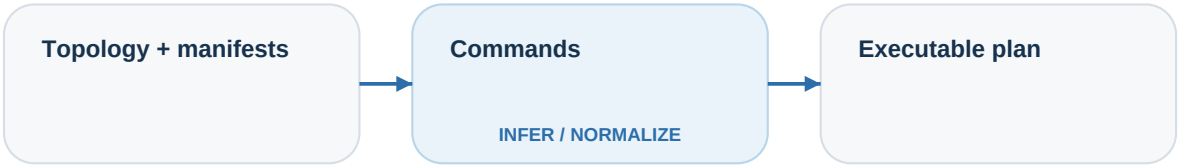
What this level tried to determine

Decide whether the repository contained one service or multiple deployable units, and infer relationships such as frontend, backend, worker, static assets, or database-owning components. This included application-unit discovery and relationship/topology inference.

Observed failure pattern

A Flask repository could contain templates and static files without being a separate frontend service. Treating those assets as another application unit changed the deployment topology even though the repository itself was valid.

Level 4 - Package, install, build, and start-command inference



What this level tried to determine

Choose the package manager, dependency-install strategy, build command, start command, and runtime entrypoint. These values converted stack classification into executable instructions.

Risk at this level

Even when framework detection was correct, a wrong command or install strategy could make the generated image fail. Because the command was normalized into the plan, later stages usually treated it as intentional rather than uncertain.

Levels 5-6: runtime and container generation

Level 5 - Port, environment, and database inference

What this level tried to determine

Determine the application port, environment ownership, database requirements, database owner component, and runtime values that should reach each application unit. This was where source understanding became runtime configuration.

Observed failure pattern

A runtime ENV change could be correct for a component while another workflow stage still checked a stale global location. The local change was valid, but the contract between stages had drifted, causing a false failure.

Level 6 - Docker template matching and Dockerfile generation

What this level tried to determine

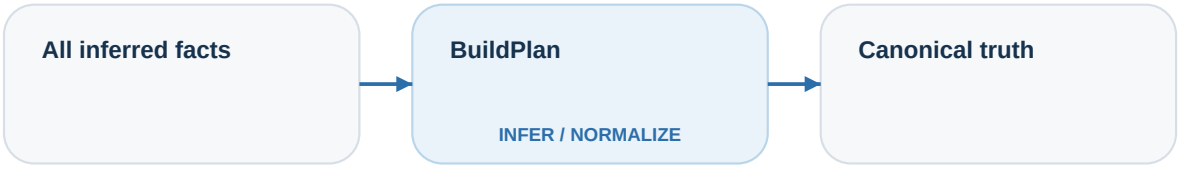
Translate the inferred stack and runtime facts into a concrete container build. The system matched templates, generated Dockerfiles, selected build context, exposed ports, and encoded inferred commands into the image build path.

Risk at this level

Container generation amplified upstream decisions. A wrong port, app root, framework, or command could be embedded into the Dockerfile, turning a detector guess into an actual build/runtime behavior.

Levels 7-9: canonical plan to AWS execution

Level 7 - BuildPlan canonicalization



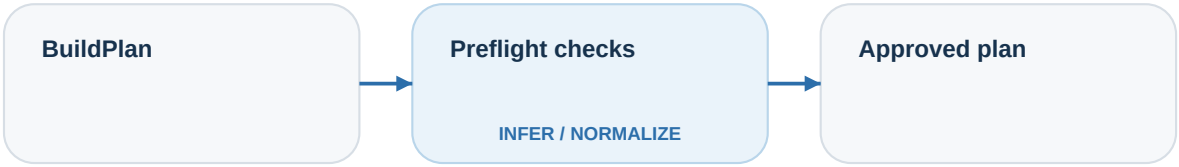
What this level tried to determine

Collect all inferred facts into one canonical deployment object: application units, topology, build context, commands, runtime, environment, database relationships, Docker configuration, and deployment profile. This became the system of record for downstream deployment.

Why this increased blast radius

Once a wrong value entered the BuildPlan, downstream code no longer saw it as a guess. It saw it as canonical state.

Level 8 - Preflight, certification, and deployability checks



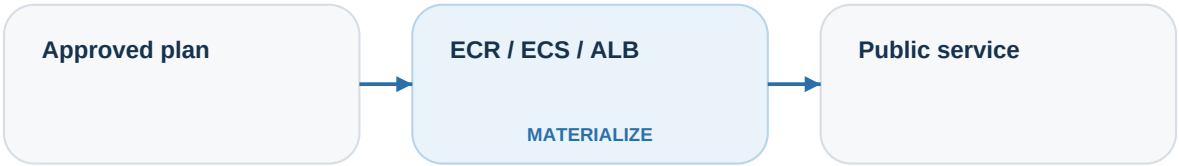
What this level tried to determine

Validate that the BuildPlan and generated artifacts appeared internally consistent before executing infrastructure. The intent was to catch defects early and prevent obviously broken deployments.

Why good checks could still fail

Preflight validated the plan it was given. If the plan contained a coherent but incorrect interpretation of the repository, validation could pass while the real application still failed.

Level 9 - AWS execution and health

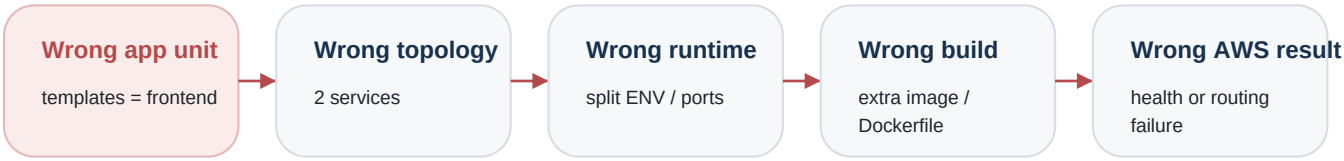


Final effect

AWS faithfully executed the plan. That is exactly why an upstream misclassification could survive until the end: infrastructure was correct relative to the plan, but the plan could be wrong relative to the application.

Why one wrong call could disrupt everything

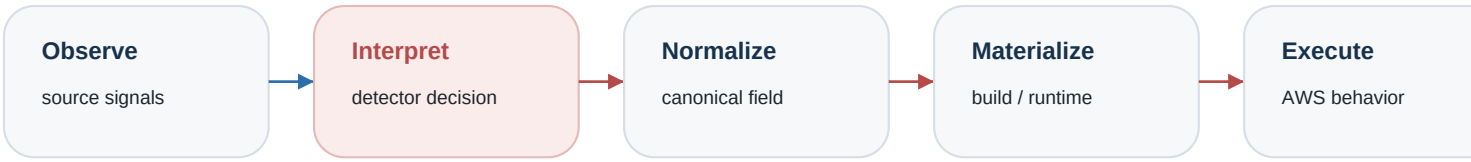
The core weakness was propagation: one incorrect interpretation could become trusted input for every later level.



Why the architecture could fail even when most stages were good

- Repository signals are ambiguous: the same files can mean different things in different application structures.
- Each level produced a definitive answer because downstream deployment needed concrete values.
- The output of one detector became the input contract of multiple later stages.
- Normalization removed uncertainty: a detector decision became a field in canonical state.
- Generated Dockerfiles, runtime ENV, ports, topology, and AWS resources physically materialized those fields.
- A preflight check could prove that the plan was internally consistent while the plan was still wrong about the real application.
- As more frameworks and repository shapes were supported, interactions between detectors increased and so did the possible blast radius.

Failure mechanism at every level



Conclusion

DeployGuard native architecture was not failing because every stage was poor. Many stages could be individually correct. The structural problem was that the system depended on a long chain of interpretation. A single wrong call at an early level could become authoritative state, pass through valid downstream logic, and finally produce a deployment that was technically consistent with the plan but inconsistent with the real application. The deeper the chain of inferred facts, the more places one wrong decision could affect.